

Metodología de la Programación

Tema 5. Sobrecarga de operadores. Completando clases

Sylvia Acid (`acid@decsai.ugr.es`)

derivado de las obras de Andrés Cano y de Juan Huete

Departamento de Ciencias de la Computación e I.A.



ugr

Universidad
de Granada



Curso 2025-26

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

eestring.h

representación

```

#ifndef __ESTRING__
#define __ESTRING__
class estring {
public:
    .....
private:
    //inicio c-string
    char *datos;
    // sig ultimo caracter
    char *fin;
    // tam bloque
    int tam;
};
#endif

```

```
estring a; // cadena vacia
```

```

a.datos - >| \0 |
a.fin   -----^
a.tam = 1

```

```

estring a("Hola");
a.datos - >| H | o | l | a | \0 |
a.fin   -----^
a.tam = 5

```

```
class estring {
    estring & assign (const estring & s);
    estring & assign (const char* s, size_t n);
}
```

- El problema de la asignación

```
int main(){
    estring s, t("Hola");
    s = t; // asignacion ...
}
```

- Se transforma en:

```
s.assign(t); // asignacion ...
```

- No sigue el estándar de asignación para los otros tipos, el operador =.
- Problema a la hora de diseñar/utilizar algoritmos, hay que diferenciar entre TDAs para la asignación (diseñador/usuario inadvertido).

Introducción a la sobrecarga de operadores

- C++ permite usar un conjunto de operadores con los tipos predefinidos que hace que el código sea muy **legible y fácil de entender**.

Introducción a la sobrecarga de operadores

- C++ permite usar un conjunto de operadores con los tipos predefinidos que hace que el código sea muy **legible y fácil de entender**.
- Por ejemplo, la expresión:

$$a + \frac{b \cdot c}{d \cdot (e + f)}$$

se calcularía en C++ con `a+(b*c)/(c*(e+f))`

Introducción a la sobrecarga de operadores

- C++ permite usar un conjunto de operadores con los tipos predefinidos que hace que el código sea muy **legible y fácil de entender**.
- Por ejemplo, la expresión:

$$a + \frac{b \cdot c}{d \cdot (e + f)}$$

se calcularía en C++ con `a+(b*c)/(c*(e+f))`

- Sin operadores para nuevas clases, utilizando funciones la expresión sería:

`Suma(a,Divide(Producto(b,c),Producto(c,Suma(e,f))))`

más engorroso de escribir y entender.

- C++ permite **sobrecargar** casi todos sus operadores (aritméticos, relacionales... tanto monarios como binarios) en nuestras clases, para que podamos reutilizarlos con los objetos de las nuevas clases.
- Para ello, definiremos un método o una función cuyo nombre estará compuesto de la palabra `operator` junto con el operador correspondiente. Ejemplo: `operator+()`.
- Esto permitirá usar la siguiente sintaxis para hacer cálculos con objetos de nuestras propias clases:

```
estring a("Hola"), b;  
b = a;  // asi suena mas como asignacion!
```

- No puede modificarse la sintaxis de los operadores (número de operandos, precedencia y asociatividad).
- No deberíamos tampoco modificar la semántica de los operadores.

Operadores que se pueden sobrecargar

+	-	*	/	%	^	&		~	«	»
=	+=	-=	*=	/=	%=	^=	&=	=	»=	«=
==	!=	<	>	<=	>=	!	&&		++	--
->*	,	->	[]	()	new	new[]	delete	delete[]		

- Los operadores que no pueden sobrecargarse son:

.	.*	::	?:	sizeof
---	----	----	----	--------

- Al sobrecargar un operador no se sobrecargan automáticamente operadores relacionados.

*p.ej., al sobrecargar + no se sobrecarga automáticamente +=,
ni al sobrecargar == lo hace automáticamente !=.*

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores**
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Sobrecarga como función externa

Sobrecarga como función externa

Consiste en añadir una **función externa** a la clase, de la forma `operator★()`, que recibirá dos objetos de la clase (o uno para operadores unarios) y devolverá el resultado de la operación.

- Ejemplo el `operator+()`

```
estring operator+(const estring & izq, const estring & dch);
```

- Cuando el compilador encuentre una expresión tal como `s+t` la interpretará como una llamada a la función `operator+(s,t)`
`s+t` $\equiv$ `operator+(s,t)`

Sobrecarga como función externa

- Podríamos sobrecargar el operador aunque los dos operandos sean de tipos distintos:

Sobrecarga como función externa

- Podríamos sobrecargar el operador aunque los dos operandos sean de tipos distintos:
 - Suma de `estring` con `* char` : `s+"aa"`
Puede ser necesario hacer sobrecarga de operadores de la forma:

```
estring operator+(const estring & izq, const char *dch)  
    ;
```

Sobrecarga como función externa

- Podríamos sobrecargar el operador aunque los dos operandos sean de tipos distintos:

- Suma de `estring` con `* char` : `s+"aa"`

Puede ser necesario hacer sobrecarga de operadores de la forma:

```
estring operator+(const estring & izq, const char *dch)
;
```

- Suma de `* char` con `estring`: `"aa"+s`

```
estring operator+(const char *izda, const estring &dcha)
;
```

Sobrecarga como función externa

`estring operator+()`

- 1 Suma de `estring` con `* char` : `s+"aa"`
- 2 Suma de `* char` con `estring`: `"aa"+s`

En la clase `estring`, ni 1 ni 2 son necesarios, porque ¿?

```
estring operator+(const estring & izq, const estring & dch){
    estring resultado(izq);
    for(int i=0; i< dch.size(); i++)
        resultado.push_back(dch[i]);
    return resultado;
}

int main(){
    estring a("Dos"),b;
    b = a+a;    // x=operator+(a,a);
    b = a+" soles";
}
```


Sobrecarga como función miembro

Sobrecarga como función miembro

Consiste en añadir **un método** a la clase, de la forma `operator★()`, que recibirá un objeto (o ninguno para operadores unarios) de la clase y devolverá el resultado de la operación.

- Ejemplo el `operator+()`

```
estring estring::operator+ (const estring & dch) const{
    estring resultado(*this);
    for(int i=0; i< dch.size(); i++)
        resultado.push_back(dch[i]);
    return resultado;
}

int main(){
    estring a("Dos"),b;
    b = a+a;    // x=a.operator+(a);
    b = a+" soles";
}
```

```
estring  estring::operator+(const estring & dch);
```

- Cuando el compilador encuentre una expresión tal como $p+q$ la interpretará como una llamada al método $p.operator+(q)$
 $p+q \equiv p.operator+(q)$
- También podríamos sobrecargar así el operador con un operando de tipo distinto:
 - Suma de estring con char *: $s+ \text{"Buenas"}$
 - Sin embargo no es posible definir así el operador para usarlo con expresiones del tipo: $\text{"Buenas"}+ s$

Sobrecarga como función miembro o externa

- La sobrecarga de un operador con una **función miembro** puede hacerse si tenemos acceso al código fuente de la clase y el primer operando es del tipo de la clase.

Sobrecarga como función miembro o externa

- La sobrecarga de un operador con una **función miembro** puede hacerse si tenemos acceso al código fuente de la clase y el primer operando es del tipo de la clase.

Sobrecarga como función miembro o externa

- La sobrecarga de un operador con una **función miembro** puede hacerse si tenemos acceso al código fuente de la clase y el primer operando es del tipo de la clase.

Ejemplo: Para concatenar 2 eststring, podemos sobrecargar `operator+` en la clase `eststring` con una función miembro pues, tenemos acceso a su implementación.

```
eststring  eststring::operator+ (const eststring & dch) const{  
    ...  
}  
  
int main(){  
    eststring a("- Buenas");  
    eststring b("- Muy buenas");  
    eststring c;  
    c = a + b; // equivalente a c = a.operator+(b);  
}
```

Sobrecarga como función miembro o externa

- La sobrecarga de un operador con una **función miembro** puede hacerse si tenemos acceso al código fuente de la clase y el primer operando es del tipo de la clase.

Ejemplo: Para concatenar 2 estring, podemos sobrecargar `operator+` en la clase `estring` con una función miembro pues, tenemos acceso a su implementación.

```
estring  estring::operator+ (const estring & dch) const{
...
}

int main(){
    estring a("- Buenas");
    estring b("- Muy buenas");
    estring c;
    c = a + b; // equivalente a c = a.operator+(b);
}
```

- El lenguaje obliga a que los operadores `()`, `[]`, `->` y los operadores de asignación (segunda fila en la tabla), sean implementados como **funciones miembro**.

Sobrecarga como función miembro o externa

- Si el primer operando debe ser un objeto de una clase diferente, debemos sobrecargarlo como **función externa**.

Ejemplo: El operador + para concatenar un `estring` con un `estring` lo implementaremos con una función externa.

```
estring operator+(const estring& cadena, const estring&cad){  
    ...  
}  
  
int main(){  
    estring p;  
    estring s1="estring: ", s2;  
    ...  
    s2 = s1 + p; // equivale a s2 = operator+(s1, p);  
}
```

- Por la misma razón, si el primer operando debe ser un dato de un tipo primitivo, debemos sobrecargarlo como **función externa**.

```
estring operator+(int i, const estring&cad){  
    ...  
}
```

```
int main(){  
    estring s1, s2;  
    int i;  
    ... // operador izda es otro tipo y no tenemos conversor  
    s1 = i + s2; // equivale a s1 = operator+(i, s2);  
}
```


Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación**
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

El operador de asignación. Un caso particular

Aunque no definido por el diseñador del TDA, el operador = se podrá utilizar...¡!

```
1 int main(){
2  estring s, t("Hola");
3  s = t;
```

S

```
s.fin -----v
s.datos 0x0010 --> | H | o | l | a | \0 |
t.datos 0x0040 ----^
t.fin -----^
t.tama = 5
```

Línea 3 (copia blanda) luego, s y t comparten la misma memoria dinámica, consecuencia: destructor de t produce un error al intentar liberar la memoria ya liberada por el destructor de s.

Luego, es necesaria la sobrecarga del operador =, cuando la clase tiene

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.
- El operador de asignación se confunde frecuentemente con el constructor de copia.

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.
- El operador de asignación se confunde frecuentemente con el constructor de copia.
 - En el constructor de copia se da valor a un objeto que está por construir.

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.
- El operador de asignación se confunde frecuentemente con el constructor de copia.
 - En el constructor de copia se da valor a un objeto que está por construir.
 - En una asignación `s=t` se da valor a un objeto que ya estaba construido (`*this` ya está construido).

El operador de asignación: primera aproximación

```
void operator=(const eststring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.
- El operador de asignación se confunde frecuentemente con el constructor de copia.
 - En el constructor de copia se da valor a un objeto que está por construir.
 - En una asignación `s=t` se da valor a un objeto que ya estaba construido (`*this` ya está construido).
 - Por ello, en el operador de asignación debemos empezar liberando la memoria dinámica alojada en `*this`.

El operador de asignación: primera aproximación

```
void operator=(const estring &s);
```

- Cuando realizamos una asignación del tipo `s=t`, el compilador lo interpreta como la llamada `s.operator=(t)`.
- Para evitar una copia innecesaria de `t`, pasamos el parámetro por `const` referencia.
- El operador de asignación se confunde frecuentemente con el constructor de copia.
 - En el constructor de copia se da valor a un objeto que está por construir.
 - En una asignación `s=t` se da valor a un objeto que ya estaba construido (`*this` ya está construido).
 - Por ello, en el operador de asignación debemos empezar liberando la memoria dinámica alojada en `*this`.
 - El resto es idéntico al constructor de copia.

El operador de asignación: primera aproximación

El formato **estándar** de un operador de asignación es

```
ClaseObjeto operator=(const ClaseObjeto & s); (1)
```

```
ClaseObjeto operator=(const ClaseOtra & x); (2)
```

Recibe un único parámetro que es un objeto del tipo que queremos asignar y devuelve un valor

```
ClaseObjeto o1,o2; ClaseOtra x;
```

```
o1 = o2; // (1) precompilador transf.: o1.operator=(o2);
```

```
o2 = x; // (2) o2.operator=(x);
```

```

estring  estring::operator= (const estring & ss){
    if(tam <= ss.tam){ // No hay espacio, necesitamos memoria
        delete [] datos; // liberamos
        datos = new char[ss.tam]; // reservamos memoria para this
        tam = ss.tam;
    }
    strcpy(datos,ss.datos);
    fin = datos+strlen(datos);
}
return *this;
}

```

- Podemos ver que coincide con el constructor de copia, excepto en las 2 primeras líneas.

```

1 int main(){
2 estring s("-Buenas"), t("-Muy buenas");
3 s = t;
4 std::cout << s.len() << s.capacity()<< std::endl;

```

El operador de asignación: segunda aproximación

- Recordemos que el operador de asignación puede usarse de la siguiente forma: `p=q=r=s;`.

```
estring& operator=(const estring & ss);
```

El operador de asignación: segunda aproximación

- Recordemos que el operador de asignación puede usarse de la siguiente forma: `p=q=r=s;`.
- C++ evalúa la expresión anterior de derecha a izquierda, de forma que lo primero que realiza es `r=s`.

```
estring& operator=(const estring & ss);
```

El operador de asignación: segunda aproximación

- Recordemos que el operador de asignación puede usarse de la siguiente forma: `p=q=r=s;`.
- C++ evalúa la expresión anterior de derecha a izquierda, de forma que lo primero que realiza es `r=s`.
- El resultado de esta última expresión (`r=s`) es el objeto que queda a la izquierda (`r`), que se usa para evaluar el siguiente operador de asignación (asignación a `q`).

```
estring& operator=(const estring & ss);
```

El operador de asignación: segunda aproximación

- Recordemos que el operador de asignación puede usarse de la siguiente forma: `p=q=r=s;`.
- C++ evalúa la expresión anterior de derecha a izquierda, de forma que lo primero que realiza es `r=s`.
- El resultado de esta última expresión (`r=s`) es el objeto que queda a la izquierda (`r`), que se usa para evaluar el siguiente operador de asignación (asignación a `q`).
- Por tanto `operator=` debe devolver el mismo tipo de la clase (estring en este caso).

```
estring& operator=(const estring & ss);
```

El operador de asignación: segunda aproximación

- Recordemos que el operador de asignación puede usarse de la siguiente forma: `p=q=r=s;`.
- C++ evalúa la expresión anterior de derecha a izquierda, de forma que lo primero que realiza es `r=s`.
- El resultado de esta última expresión (`r=s`) es el objeto que queda a la izquierda (`r`), que se usa para evaluar el siguiente operador de asignación (asignación a `q`).
- Por tanto `operator=` debe devolver el mismo tipo de la clase (estring en este caso).
- Para que la llamada a `r.operator=(s)` devuelva el objeto `r` es necesario que la devolución sea por referencia.

```
estring& operator=(const estring & ss);
```


El operador de asignación: segunda aproximación

```
estring::estring & operator= (const estring & s){
    if(tam <= s.tam){ // No hay espacio, necesitamos memoria
        delete [] datos; // liberamos
        datos = new char[s.tam]; // reservamos memoria para this
        tam = s.tam;
    }
    strcpy(datos,s.datos);
    fin = datos+strlen(datos);
}
return *this;
}
```

- Como podemos ver, el método devuelve (por referencia) el objeto actual.

El operador de asignación: implementación final

```
estring& operator=(const estring &s);
```

- En el caso de realizar una asignación del tipo `s=s` nuestro operador de asignación no funcionaría bien.
- En tal caso, dentro del método `operator=`, `*this` y `s` son el mismo objeto.

```
estring::estring & operator= (const estring & s){ {
    if (this != &s) {      // si s=s no hace nada
        if(tam <= s.tam){  // No hay espacio, necesitamos memoria
            delete [] datos;      // liberamos
            datos = new char[s.tam]; // reservamos memoria para this
            tam = s.tam;
        }
        strcpy(datos,s.datos);
        fin = datos+strlen(datos);
    }
    return *this;
}
```

El operador de asignación: esquema genérico

```
Clase & operator=(const Clase &p);
```

- En una clase que tenga datos miembro que usen memoria dinámica, éste sería el esquema genérico que debería tener operator=.

```
CLASE& CLASE::operator=(const CLASE &p)
{
    if (&p!=this) { // Si no es el mismo objeto
        // Si *this tiene memoria dinamica -> liberarla
        // Copiar p en *this (reservar nueva memoria y copiar)
    }
    return *this; // Devolver referencia a *this
}
```

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima**
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

La clase mínima

- La clase, normalmente se dota de un *constructor por defecto*.
- Clase con datos miembro que usan memoria dinámica, se dota de: *destructor, constructor de copia y operador de asignación*.

```
class estring {  
    char *datos;  
    char *fin;  
    int tam;  
public:  
    estring();  
    estring(const estring &orig);  
    estring (size_t n, char c);  
    estring & operator= (const estring & s);  
    ~estring();  
    ...  
};
```

Funciones miembro predefinidas

C++ proporciona una implementación por defecto para el constructor por defecto, destructor, constructor de copia y operador de asignación.

- Si no incluimos ningún constructor, C++ proporciona el **constructor por defecto** que tiene un cuerpo vacío.

Funciones miembro predefinidas

C++ proporciona una implementación por defecto para el constructor por defecto, destructor, constructor de copia y operador de asignación.

- Si no incluimos ningún constructor, C++ proporciona el **constructor por defecto** que tiene un cuerpo vacío.
- Si no incluimos el **destructor**, C++ proporciona uno con cuerpo vacío.

Funciones miembro predefinidas

C++ proporciona una implementación por defecto para el constructor por defecto, destructor, constructor de copia y operador de asignación.

- Si no incluimos ningún constructor, C++ proporciona el **constructor por defecto** que tiene un cuerpo vacío.
- Si no incluimos el **destructor**, C++ proporciona uno con cuerpo vacío.
- Si no incluimos el **constructor de copia**, C++ proporciona uno que hace una copia de cada dato miembro llamando al constructor de copia de la clase a la que pertenece cada uno.

Funciones miembro predefinidas

C++ proporciona una implementación por defecto para el constructor por defecto, destructor, constructor de copia y operador de asignación.

- Si no incluimos ningún constructor, C++ proporciona el **constructor por defecto** que tiene un cuerpo vacío.
- Si no incluimos el **destructor**, C++ proporciona uno con cuerpo vacío.
- Si no incluimos el **constructor de copia**, C++ proporciona uno que hace una copia de cada dato miembro llamando al constructor de copia de la clase a la que pertenece cada uno.
- Si no incluimos el **operador de asignación**, C++ proporciona uno que hace una asignación de cada dato miembro de la clase.

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>**
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Sobrecarga del operador <<

Operador de salida, <<

Se usa para enviar el contenido de un objeto a un flujo de salida (por ej. cout)

Sobrecarga del operador <<

Operador de salida, <<

Se usa para enviar el contenido de un objeto a un flujo de salida (por ej. `cout`)

- Podemos sobrecargar el operador << para mostrar un objeto usando la sintaxis `cout << s`
`cout << s` $\equiv$ `cout.operator<<(s)`

Sobrecarga del operador <<

Operador de salida, <<

Se usa para enviar el contenido de un objeto a un flujo de salida (por ej. `cout`)

- Podemos sobrecargar el operador << para mostrar un objeto usando la sintaxis `cout << s`
`cout << s` $\equiv$ `cout.operator<<(s)`
- No podemos añadir un método a la clase `ostream` (a la que pertenece `cout`) luego, usamos una **función externa**.

```
std::ostream & operator<<(std::ostream &, const estring &);
```

```
std::ostream & operator<<(std::ostream& os, const estring& cad){  
    os << cad.data() << " caben:" << cad.capacity();  
    return os;  
}
```

Al ser externa no tiene acceso a los datos!! luego, ha de usar los métodos públicos. Otra opción es declararla como función friend dentro de la clase.

```
friend std::ostream & operator<<(std::ostream &, const estring  
&);
```

```
std::ostream & operator<<(std::ostream& os, const estring& cad){  
    os << cad.datos << " caben:" << cad.tam;  
    return os;  
}
```

Sobrecarga del operador <<

- La función hace una devolución por referencia del flujo (`ostream&`).

Sobrecarga del operador <<

- La función hace una devolución por referencia del flujo (ostream&).
- Esto se hace para poder usar sentencias como :

```
estring s1("-Buenas"), s2("-Muy buenas");  
    cout << s1 << s2 << "-Tiene gambas?" << s2;  
}
```


Sobrecarga del operador <<

- La función hace una devolución por referencia del flujo (ostream&).
- Esto se hace para poder usar sentencias como :

```
estring s1("-Buenas"), s2("-Muy buenas");  
    cout << s1 << s2 << "-Tiene gambas?" << s2;  
}
```

- cout << s1 << s2 se evalúa de izquierda a derecha:
(cout << s1) << s2;
 (cout << s2);

Sobrecarga del operador >>

Operador >> (operador de entrada)

Se usa para leer el contenido de un objeto desde un flujo de entrada (por ej. `cin`).

- Podemos sobrecargar el operador `>>` para leer un objeto usando la sintaxis `cin >> p` (equivalente a `cin.operator>>(p)`).
- De nuevo, puesto que no podemos añadir un método a la clase `istream` (a la que pertenece `cin`), sobrecargaremos este operador con una **función externa**, `friend` (o no).

Sobrecarga del operador >>

```
friend std::istream & operator>>(std::istream &,  estring &);
```

```
std::istream & operator>>(std::istream & is,  estringMP & cad){  
    char c;  
    cad.clear(); // limpia el objeto anterior  
    c=is.get();  // lectura adelantada  
    while (c != ' ' && c != '\t' && c != '\n') { //separadores  
        cad.push_back(c); // aniade en la cadena  
        c=is.get();  
    }  
    is.unget(); // repone el separador  
    return is;
```

- De nuevo, el método devuelve por referencia el flujo (istream&).

Sobrecarga del operador >>

```
friend std::istream & operator>>(std::istream &,  estring &);
```

```
std::istream & operator>>(std::istream & is,  estringMP & cad){  
    char c;  
    cad.clear(); // limpia el objeto anterior  
    c=is.get();  // lectura adelantada  
    while (c != ' ' && c != '\t' && c != '\n') { //separadores  
        cad.push_back(c); // aniade en la cadena  
        c=is.get();  
    }  
    is.unget(); // repone el separador  
    return is;
```

- De nuevo, el método devuelve por referencia el flujo (istream&).
- Esto se hace para poder usar sentencias como las siguientes:

```
estring s1, s2;  
cin >> s1;  
cin >> s1 >> s2;
```

Sobrecarga del operador >>

```
friend std::istream & operator>>(std::istream &,  eststring &);
```

```
std::istream & operator>>(std::istream & is,  eststringMP & cad){  
    char c;  
    cad.clear(); // limpia el objeto anterior  
    c=is.get();  // lectura adelantada  
    while (c != ' ' && c != '\t' && c != '\n') { //separadores  
        cad.push_back(c); // aniade en la cadena  
        c=is.get();  
    }  
    is.unget(); // repone el separador  
    return is;
```

- De nuevo, el método devuelve por referencia el flujo (istream&).
- Esto se hace para poder usar sentencias como las siguientes:

```
eststring s1, s2;  
cin >> s1;  
cin >> s1 >> s2;
```

Sobrecarga del operador >>: Ejemplo

```
Pelotas original;  
cin >> original; // sobrecarga de >> con objetos Pelotas  
Pelota p1, p2;  
cin >> p1; // sobrecarga de >> con objetos pelota  
cin >> p1 >> p2;
```

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación**
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Operador de indexación

Operador de indexación

La función `operator[]` permite sobrecargar el operador de indexación.

- Debe realizarse usando un método de la clase con un parámetro (índice) que podría ser de cualquier tipo.

Operador de indexación

Operador de indexación

La función `operator[]` permite sobrecargar el operador de indexación.

- Debe realizarse usando un método de la clase con un parámetro (índice) que podría ser de cualquier tipo.
- De esta forma podremos cambiar la sintaxis:

```
c = s.at(i);
```

por esta otra:

```
c = s[i];
```

Pero también :

```
s[i] = 'a';
```

Operador de indexación

Así necesitamos usar & en su versión constante y no constante como se hizo con el método at():

```
char& operator[] (size_t pos);  
const char &operator[] (size_t pos) const;
```

```
char& eststring::operator[] (size_t pos){  
    return datos[pos];  
}  
const char& eststring::operator[] (size_t pos) const{  
    return datos[pos];  
}
```

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos**
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Operadores de asignación compuestos

Operadores de asignación compuestos

Son los operadores `+=`, `-=`, `*=`, `/=`, `%=`, `^=`, `&=`, `|=`, `»=`, `«=`

- Tener implementado el operador `+` y el operador `=` no supone la existencia automática del operador `+=`, y así con el resto: debemos implementarlo de **forma explícita**.
- Obligatoriamente deben implementarse como método de la clase.
- Estos operadores deben devolver una referencia al objeto usado en la llamada, para así poder hacer llamadas como:

```
s3 = (s1 += s2);
```

implementación:

```
estring& estring::operator+=(const estring&cad){
    (*this) = (*this) + cad;
    return *this;
}
```

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales**
- 9 Operadores de incremento y decremento
- 10 Operador de llamada a función

Operadores relacionales

Operadores relacionales

Son los operadores binarios `==`, `!=`, `<`, `>`, `<=` y `>=`, que devuelven un valor booleano.

- Se usan cuando es necesario establecer una *relación de orden* entre los objetos de la clase.
- El definir una parte de los operadores no implica que los demás lo estén de forma automática.

Ejemplo: si definimos el operador `==`, el operador `!=` no estará definido de forma automática.

Operadores relacionales

Ejemplo: operador == en estring

- `cad1 == cad2` si ambos tienen la misma longitud y son iguales caracter a caracter.

Operadores relacionales

Ejemplo: operador == en estring

- `cad1 == cad2` si ambos tienen la misma longitud y son iguales caracter a caracter.

```
bool operator==(const estring & izq, const estring & dch){  
    bool igual = true;  
    if ( izq.size()!=dch.size() ) igual = false;  
    else {  
        for (int i=0; i<izq.size() && igual; i++)  
            if (izq[i]!=dch[i]) igual = false;  
    } // aviso: puede salir D rango en lectura  
    return igual;  
}
```


Operadores relacionales

El resto de operadores relacionales

```
bool operator<(const estring & izq, const estring & dch);  
bool operator>(const estring & izq, const estring & dch);  
bool operator<=(const estring & izq, const estring & dch);  
bool operator>=(const estring & izq, const estring & dch);
```

Operadores relacionales

El resto de operadores relacionales

```
bool operator<(const estring & izq, const estring & dch);
bool operator>(const estring & izq, const estring & dch);
bool operator<=(const estring & izq, const estring & dch);
bool operator>=(const estring & izq, const estring & dch);
```

Todos los operadores relacionales pueden definirse a partir del operador <.

```
bool operator != (const estring &cad1, const estring &cad2){
    return cad1<cad2 || cad2<cad1; }
```

$a == b$	$!(a < b \parallel b < a)$
$a > b$	$b < a$
$a != b$	$a < b \parallel b < a$
$a <= b$	$!(b < a)$
$a >= b$	$!(a < b)$

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento**
- 10 Operador de llamada a función

Operadores de incremento y decremento ($++$ y $--$)

Operadores de incremento y decremento

Son operadores unarios que tienen dos versiones: ***pre*** y ***post***, tanto para incremento como para decremento.

Suponiendo que tuviésemos definido el método `operator+(int i)` podríamos definir el incremento como

Operadores de incremento y decremento ($++$ y $--$)

Operadores de incremento y decremento

Son operadores unarios que tienen dos versiones: *pre* y *post*, tanto para incremento como para decremento.

Suponiendo que tuviésemos definido el método `operator+(int i)` podríamos definir el incremento como

```
estring& estring::operator++(){
    *this = *this + 1;
    return *this;
}

estring& estring::operator--(){
    *this = *this - 1;
    return *this;
}

int main(){
    estring cad;
    ...
    ++cad;
    ...
    --cad;
```

Operadores de incremento y decremento ($++$ y $--$)

Operadores de posincremento y posdecremento

Los nombres de las funciones para los operadores *pos* coinciden con los *pre*.

Por ello, el estándar de C++ propone que:

- Cuando el compilador encuentra $++obj$, se genera una llamada a `obj.operator++()`.
- Cuando el compilador encuentra $obj++$, se genera una llamada a `obj.operator++(0)`. En este caso se añade un valor entero a la llamada, que no se usa para nada, pero que sirve para distinguirla de la anterior.

Operadores de incremento y decremento (++ y --)

```
estring estring::operator++(int valor){
    estring aux(*this);
    *this = *this + 1;
    return aux;
}

estring estring::operator--(int valor){
    estring aux(*this);
    *this = *this - 1;
    return aux;
}

int main(){
    estring cad;
    ...
    cad++;
    ...
    cad--;
}
```

¡Cuidado!

La devolución en este caso debe hacerse por valor. ¿Por qué?

Contenido del tema

- 1 Sin los operadores, un TDA es menos...
- 2 Mecanismos de sobrecarga de operadores
 - Sobrecarga como función externa
 - Sobrecarga como función miembro
 - Una cuestión de diseño: operador como método o función
- 3 El operador de asignación
- 4 La clase mínima
- 5 Operadores << y >>
 - Sobrecarga del operador <<
 - Sobrecarga del operador >>
- 6 Operador de indexación
- 7 Operadores de asignación compuestos
- 8 Operadores relacionales
- 9 Operadores de incremento y decremento
- 10 **Operador de llamada a función**

Operador de llamada a función

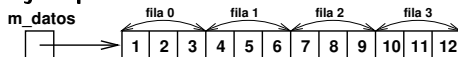
Operador de llamada a función

Es la función `operator()` que obligatoriamente se implementará como función miembro.

Puede implementarse con cualquier número de parámetros (podemos tener varias versiones de este operador).

Operador de llamada a función

Ejemplo:



```
class Matriz {
    double* m_datos;
    int m_filas, m_columnas;
public:
    Matriz(int nf, int nc){
        m_filas=nf;
        m_columnas=nc;
        m_datos = new double[m_filas*m_columnas];
    }
    double& operator() (int fila, int columna){
        assert(fila>=0 && fila<m_filas && columnas >=0 &&
columna<m_columnas);
        return m_datos[fila*m_columnas + columna];
    }
    // sigue
```

Operador de llamada a función

```
class Matriz {
    double* m_datos;
    int m_filas, m_columnas;
public:
    ...
    const double& operator() (int fila, int columna) const{
        assert(fila>=0 && fila<m_filas && columnas >=0 &&
            columna<m_columnas);
        return m_datos[fila*m_columnas + columna];
    }
}

int main(){
    Matriz m(4,3);
    ...
    cout<<m(3,2)<<endl;
    m(3,2) = 7.4;
}
```

Recapitulando

Directrices para elegir entre miembro y no-miembro

Según el libro de Rob Murray, C++ Strategies & Tactics , Addison Wesley, 1993, página 47.

Operador	Uso recomendado
Todos los operadores unarios	miembro
= () [] -> ->*	debe ser miembro
+= -= /= *= ^= &= = %= >>= <<=	miembro
El resto de operadores binarios	no miembro

- Sin la sobrecarga del operador `=`, no se podría utilizar la asignación entre objetos de nuestra nueva clase. Qué opinas?
- Se podría definir un método `at(int i)` que devuelva una referencia no constante?
- Porqué el operador de preincremento devuelve el objeto por referencia?
- Porqué el operador de postincremento devuelve el objeto por valor ?
- Cuando se define `char &operator[] (size_t pos) const;` qué es constante?
- Se puede definir un método
`char &operator[] (size_t pos) const;?`
- Se puede definir un método
`const char &operator[] (size_t pos);?`

- Sea `ClaseA p;` donde la clase tiene memoria dinámica
 - En la ejecución de `p = p + 5` qué funciones o métodos se llaman?
 - Para realizar la operación `p = p + 5` se libera espacio?
 - En la ejecución de `p = 5 + p` qué funciones o métodos se llaman?
 - Supongamos `p = p + 5.2` y tan solo se considera entero como argumento. Indica qué ocurre.

